AGENCYOS

Requirements, Scope & Change Request System

Document 11 of the AgencyOS Master Documentation Set

REQUIREMENT COLLECTION • FEATURE EXTRACTION • SCOPE FREEZE • FREE/SMALL CHANGES • PAID/LARGE CHANGES • REVISIONS • APPROVALS

Purpose: Define the system that converts client conversations and approved commercial requirements into a controlled project scope, prevents scope drift, and manages every requested change through a traceable workflow.

1. Core Principle

> The approved scope is the contract-like operational boundary for project delivery. AI agents may interpret and execute the scope, but they must not silently expand, reduce or rewrite it.

- Requirements are collected before scope is finalized.
- Confirmed requirements are converted into structured features.
- The approved quotation and scope are linked.
- Scope is versioned and frozen at the agreed stage.
- Client requests after scope freeze are classified before work begins.
- Small/free changes follow configured limits.
- Large/paid changes require a change request and, where applicable, revised quotation.
- Every approved change creates a new scope version.
- Old scope versions remain historical and auditable.

2. End-to-End Requirements Lifecycle

> CONVERSATION → REQUIREMENT EXTRACTION → CLARIFICATION → REQUIREMENT CONFIRMATION → FEATURE BREAKDOWN → ESTIMATION → QUOTATION → CLIENT ACCEPTANCE → SCOPE FREEZE → DELIVERY → CHANGE REQUEST → IMPACT ANALYSIS → APPROVAL → NEW SCOPE VERSION → IMPLEMENTATION

The system must distinguish between what the client said, what the AI inferred, what the agency proposed, what the client confirmed, and what was finally approved.

3. Requirement Sources

- Sales WhatsApp conversation.
- Website/form submissions.
- Email or other configured channels.
- Voice notes and transcripts.
- Documents/briefs supplied by client.
- Portfolio/reference discussions.
- Client calls/meeting notes where integrated.
- Existing project/change requests.
- Admin-approved internal requirements.

Every material requirement should have a source reference or explicit confirmation status.

4. Requirement Record

Required fields

- Requirement ID.
- Client ID.
- Project ID where applicable.
- Source message/document/event.
- Requirement text.
- Normalized description.
- Category.

• Priority.
• Status.
• Confirmed by client.
• Confirmed timestamp.
• Assumptions.
• Dependencies.
• Acceptance criteria.
• Related feature IDs.
• Created/updated timestamps.

5. Requirement Status

• CAPTURED — extracted from source but not confirmed.
• CLARIFICATION_REQUIRED — material information is missing.
• CONFIRMED — client has confirmed the requirement.
• REJECTED — client/agency rejected it.
• CONVERTED_TO_SCOPE — included in approved scope.
• OUT_OF_SCOPE — intentionally excluded.
• SUPERSEDED — replaced by a newer requirement/version.
• CANCELLED — withdrawn.

> An AI extraction must never automatically become CONFIRMED simply because the model is confident.

6. AI Requirement Extraction

• Read relevant conversation/context.
• Extract explicit requirements.
• Separate facts from assumptions.
• Identify ambiguous statements.
• Identify duplicate requirements.
• Identify dependencies.
• Identify missing information.
• Suggest acceptance criteria.
• Assign provisional categories/priorities.
• Link each extraction to source evidence.

The AI should assist the Sales/PM workflow, not manufacture requirements.

7. Requirement Clarification

> If a feature materially affects price, timeline, architecture or acceptance criteria and the information is unclear, the system should ask a focused clarification question.

• Avoid asking for information already present in memory.
• Group related questions.
• Prefer simple client-facing language.
• Record the client's answer.

• Update the requirement version.

• Re-evaluate downstream impact when clarification changes scope.

8. Feature Extraction

Convert requirements into implementation-ready feature units

• Feature ID.

• Feature name.

• Description.

• User/business purpose.

• Priority.

• Platforms.

• Dependencies.

• Acceptance criteria.

• UI impact.

• Backend/API impact.

• Database impact.

• Testing requirements.

• Estimated complexity.

• Included/excluded status.

• Related requirement IDs.

A feature is a delivery unit. A requirement may generate multiple features, and one feature may support multiple related requirements.

9. Scope Structure

Scope hierarchy

• Project.

• Module/area.

• Feature.

• Sub-feature/task.

• Acceptance criteria.

• Exclusions.

• Dependencies.

• Assumptions.

• Deliverables.

• Milestone.

> The scope must be detailed enough for PM, UI, development and QA agents to operate without repeatedly returning to the original sales conversation.

10. Included vs Excluded

• INCLUDED — explicitly part of approved scope.

• EXCLUDED — explicitly not part of scope.

• PENDING — requires clarification/decision.

• OPTIONAL — discussed but not included unless selected.

• CHANGE REQUEST — requested after freeze or outside the approved boundary.

Explicit exclusions are important. They prevent a later conversation from being interpreted as an accidental commitment.

11. Scope Estimation

• Estimate feature complexity.

• Estimate UI effort.

• Estimate backend/API effort.

• Estimate integration effort.

• Estimate testing effort.

• Estimate deployment effort.

• Identify dependencies/risks.

• Estimate timeline impact.

• Estimate commercial impact.

AI may generate estimates, but committed commercial/timeline values remain subject to configured policy and approval.

12. Scope and Quotation Link

> Every approved quotation should reference an exact scope version, and every scope version should identify the quotation/change request that authorized it.

• Quote V1 → Scope V1.

• Revised Quote V2 → Scope V2 after approved change.

• Change request without commercial impact may still create a new scope version where required.

• Rejected change does not alter active scope.

• Superseded scope remains historical.

13. Scope Freeze

> SCOPE DISCOVERY → CLIENT CONFIRMATION → QUOTATION → ACCEPTANCE → SCOPE FREEZE

• Freeze means the active scope becomes the delivery baseline.

• Agents must compare new requests against the frozen scope.

• The system should show scope version and freeze timestamp.

• Only authorized change workflow can modify the active baseline.

• Emergency/admin corrections must be versioned and audited.

Scope freeze does not mean the project can never change. It means changes must become explicit change requests.

14. Scope Freeze Checklist

• Requirements confirmed.

• Features extracted.

• Dependencies identified.

• Acceptance criteria defined.

• Exclusions recorded.

• Timeline assumptions recorded.

• Commercial scope aligned with quote.
• Client acceptance recorded.
• Active scope version created.
• Scope freeze timestamp recorded.

15. Change Request Trigger

• Client requests a new feature.
• Client expands an existing feature.
• Client changes approved behavior.
• Client adds a new platform.
• Client adds an integration.
• Client changes a design direction materially.
• Client requests additional screens/flows.
• Client requests additional reports/admin features.
• Client asks for work explicitly excluded from scope.
• Client requests additional revisions beyond the configured allowance.

> The system should detect likely change requests automatically, but the final classification must follow deterministic rules/policy.

16. Change Request Record

• Change Request ID.
• Project ID.
• Client ID.
• Request source.
• Original message/evidence.
• Requested change.
• Related requirement/feature.
• Current scope version.
• Classification.
• Impact estimate.
• Price impact.
• Timeline impact.
• Risk impact.
• Decision.
• Approval status.
• Quote version.
• New scope version.
• Requester.
• Created/updated timestamps.

17. Change Classification

Class	Meaning	Action

Class	Meaning	Typical action
IN_SCOPE	Already covered by active scope	Implement normally
SMALL/FREE	Minor change within configured allowance	Approve under policy
PAID_CHANGE	Material additional work	Re-estimate + revised quote + approval
NEW_PROJECT	Large/new objective beyond project boundary	Create new commercial/project workflow
CLARIFICATION	No actual scope change; meaning unclear	Clarify
DUPLICATE	Already requested/approved	Link to existing item
REJECTED	Not accepted or not feasible	Record reason

18. Small / Free Change Policy

> Small/free changes are allowed only within explicit configured boundaries. 'Small' must be a policy definition, not an agent's personal judgment.

Possible criteria

• Limited UI text/label correction.

• Minor visual adjustment.

• Small content change.

• Minor configuration adjustment.

• Small correction that does not change architecture.

• Change below configured effort threshold.

• Change within configured revision allowance.

• No material timeline or dependency impact.

Exact thresholds are configurable by project/service type.

19. Free Revision Allowance

• Configure included design revisions.

• Configure included prototype revisions.

• Configure included minor change requests.

• Track used/remaining allowance.

• Define what counts as one revision.

• Define when multiple edits count as one consolidated revision.

• Prevent agents from resetting the counter without authorization.

> The client should clearly know what is included and when additional work becomes chargeable.

20. Large / Paid Change

> REQUEST → CLASSIFY → IMPACT ANALYSIS → ESTIMATE → REVISED QUOTE → ADMIN APPROVAL WHERE REQUIRED → CLIENT ACCEPTANCE → PAYMENT CONDITION → NEW SCOPE VERSION → IMPLEMENTATION

• Do not start material paid work before the required commercial gate.

• Link paid change to quote/invoice.

• Record payment status separately.

• Update timeline if approved.

• Update milestone/task plan.

• Preserve previous scope version.

21. Change Impact Analysis

Analyze

• UI impact.

• Prototype impact.

• Frontend impact.

• Backend/API impact.

• Database impact.

• Integration impact.

• QA impact.

• Security impact.

• Performance impact.

• Deployment impact.

• Timeline impact.

• Cost impact.

• Dependencies.

• Risk.

The PM Agent coordinates impact analysis with specialist agents.

22. Change Request State Machine

> DRAFT → SUBMITTED → TRIAGE → ANALYZING → CLASSIFIED → QUOTE_REQUIRED / FREE_APPROVAL / REJECTED → CLIENT_APPROVAL → PAYMENT_GATE → APPROVED → IMPLEMENTING → QA → COMPLETED

• A request may move to CLARIFICATION_REQUIRED at any stage before final approval.

• A rejected request does not alter active scope.

• A client-approved paid change still requires the configured payment condition before implementation.

• Implementation completion must be verified independently.

23. Client Change Request Workflow

• Client sends request in WhatsApp.

• System links request to project.

• AI extracts requested change.

• PM/Policy Engine classifies it.

• If in scope, normal task is created.

• If small/free, apply configured allowance.

• If paid, create change request and impact analysis.

• Generate revised quote.

• Obtain required Admin/client approval.

• Create new scope version.

• Schedule implementation.

• QA the change.

• Close change request.

24. Change Request Communication

• Tell client whether request is already in scope, a small included change, or a paid change.

• Avoid technical jargon unless client asks.

• For paid changes, clearly state added cost and timeline impact.

• For rejected changes, give a professional reason.

• For clarification, ask the minimum necessary question.

• Do not promise implementation before required approval/payment.

All client-facing commercial statements must remain within the Policy Engine.

25. Revision Management

Revision types

• UI design revision.

• Prototype revision.

• Content revision.

• Functional revision.

• Scope revision.

• Technical revision.

• Client-requested revision.

• Agency correction.

Agency corrections that fix an agency-caused defect should not automatically consume a client's included revision allowance.

26. Revision Rules

• Track revision number.

• Track requested changes.

• Track scope impact.

• Track whether revision is included.

• Track approval.

• Track artifact version.

• Track client feedback.

• Track completion.

• Prevent indefinite revision loops.

> A revision should always identify what changed from the previous version.

27. Design Revision Example

> UI V1 → Client feedback → UI V2 → Client feedback → UI V3 → Approved

• If revisions remain within included allowance, no extra commercial change is required.

• If the client requests a materially different product direction after approval/freeze, create a change request.

• If the change is agency correction against approved requirements, treat it as defect/correction according to policy.

28. Prototype Revision Example

• Prototype V1 delivered.

• Client identifies minor interaction issue → included correction.

• Client asks for a new workflow not in scope → change request.

• Client approves Prototype V2.

• Development begins against the approved prototype version.

Development agents must reference the approved prototype/version rather than an obsolete design.

29. Scope Versioning

• Scope V1 — initial approved scope.

• Scope V2 — approved change.

• Scope V3 — another approved change.

• Each version contains complete active scope or a deterministic delta plus its baseline.

• Version has effective timestamp.

• Version has source quote/change request.

• Version has approval evidence.

• Version has author/actor.

• Old versions remain read-only history.

30. Preventing Scope Drift

• Compare task against active scope.

• Compare feature against approved scope.

• Flag tasks with no scope mapping.

• Flag code/features with no approved requirement where detectable.

• Require change request for material unmatched work.

• Do not allow agents to treat conversational suggestions as approved scope.

• Show PM a scope-drift dashboard.

> A project can only be reliably automated if the system knows what it is supposed to build.

31. Scope Drift Detection

Potential signals

• Task has no requirement/feature mapping.

• Feature appears in implementation but not active scope.

• Client asks for excluded feature.

• Estimate materially exceeds approved scope.

• New platform appears.

• New integration appears.

• New user role/workflow appears.

• Artifact contains screens not represented in scope.

• Repeated revisions indicate a changed product direction.

Detection should create a review/flag, not automatically accuse either party.

32. Change Request Pricing

• Use approved pricing rules.

• Estimate incremental effort.

• Apply configured rate/complexity rules.

• Apply approved discounts only.

• Include tax/GST treatment where applicable.

• State added cost clearly.

• State payment requirement.

• State quote validity.

• Create revised quote version.

> AI can prepare the calculation; the Policy Engine determines whether the resulting price is allowed.

33. Change Request Timeline

• Estimate added effort.

• Identify dependency changes.

• Estimate QA/retest effort.

• Estimate deployment impact.

• Estimate client review time.

• Update target date only after approved change.

• Record original and revised target dates.

• Record reason for schedule change.

Never silently extend the deadline after accepting a scope change.

34. Approval Rules

• Free/small change can auto-approve only within configured threshold.

• Paid change requires client acceptance.

• Discounted paid change may require Admin approval.

• Large change may require Admin approval.

• Change affecting production/security may require elevated approval.

• Change affecting payment milestones may require Finance/Admin approval.

• New-project classification starts a new commercial workflow.

• Policy version is recorded.

35. Change Request Payment Gate

> APPROVED CHANGE → INVOICE/PAYMENT REQUEST → PAYMENT VERIFIED (IF REQUIRED) → IMPLEMENTATION UNLOCKED

• Payment proof is not equivalent to payment verification.

• If an approved exception allows work before payment, record the exact exception.

• Implementation must remain within the approved change scope.

• Additional requests during the change follow the same classification process.

36. Admin Controls

• Configure free-change thresholds.
• Configure revision allowances.
• Configure paid-change rules.
• Configure pricing/complexity rules.
• Approve exceptions.
• Reject change requests.
• Override supported classification with reason.
• Approve revised quotes.
• Approve scope versions.
• View scope history.
• View drift alerts.
• Pause change automation.
• Audit every material decision.

37. PM Agent Responsibilities

• Maintain active scope.
• Review new client requests.
• Coordinate classification.
• Request specialist impact analysis.
• Prepare change request.
• Coordinate quote.
• Track approval/payment gates.
• Create new scope version.
• Update tasks/milestones.
• Communicate status to client.
• Verify implementation and closure.

> PM Agent is the scope guardian: it should prevent both accidental scope expansion and unnecessary bureaucracy for genuinely small included changes.

38. Specialist Agent Responsibilities

UI Agent

• Identify screen/flow impact.
• Update design only after approved change.

Developer Agents

• Estimate implementation impact.
• Do not implement unapproved material scope.

QA Agent

• Add change-specific tests.
• Verify only approved behavior.

Finance Agent

• Create invoice/payment request for approved paid change.

Orchestrator

• Route analysis tasks and enforce agent permissions.

39. Change Request Memory & Audit

• Original client request.

• AI interpretation.

• Clarification.

• Classification.

• Impact analysis.

• Quote versions.

• Admin approval.

• Client acceptance.

• Payment status.

• Scope version.

• Implementation.

• QA result.

• Closure.

This connects to the shared memory system so future agents know why a feature exists and which version was approved.

40. Reporting

• Number of change requests.

• Free vs paid changes.

• Change-request revenue.

• Average change value.

• Average approval time.

• Average implementation time.

• Most common change categories.

• Most common scope-drift signals.

• Revision usage by project.

• Projects with excessive revisions.

• Client change frequency.

• Agency correction rate.

• Change impact on timelines.

• Change request rejection rate.

41. Acceptance Criteria

• Requirements can be captured from conversations and documents.

• AI can extract requirements with source evidence.

• Requirements have explicit confirmation status.

• Features can be derived from confirmed requirements.

• Scope has versioning.

• Quotation and scope are linked.

- Scope can be frozen.
- New client requests are classified.
- Small/free changes use configurable rules.
- Revision allowances are tracked.
- Large/paid changes create structured change requests.
- Impact analysis covers cost, time, technical and QA effects.
- Paid changes use revised quotations where required.
- Client approval is recorded against the exact change.
- Payment gates can block implementation.
- Approved changes create a new scope version.
- Old scope remains auditable.
- Scope drift can be detected/flagged.
- Admin can configure and override supported policies.
- PM coordinates change management.
- Agents cannot silently expand scope.
- Reports show change/revision patterns.
- Change history is available to future agents.

42. Final Architecture

> CONVERSATION → REQUIREMENTS → FEATURES → QUOTE → APPROVAL → SCOPE FREEZE → DELIVERY → CHANGE REQUEST → CLASSIFY → IMPACT → QUOTE/APPROVAL → PAYMENT GATE → NEW SCOPE VERSION → IMPLEMENT → QA → CLOSE

The Requirements, Scope & Change Request System protects the agency from uncontrolled scope while preserving a good client experience. The client can ask for changes naturally through WhatsApp; AgencyOS turns those requests into clear, explainable decisions.

> The key rule: no important work should exist without a reason, no important requirement without a source, no scope change without a record, and no paid work without the required commercial approval.